

# Remove std::weak\_equality and std::strong\_equality

# 

Document #: P1959R0  
Date: 2019-11-07  
Project: Programming Language C++  
EWG, LEWG  
Reply-to: Barry Revzin  
<[barry.revzin@gmail.com](mailto:barry.revzin@gmail.com)>

## § 1 Introduction

---

This paper resolves NB comments [US 170](#):

The `strong_equality` and `weak_equality` comparison categories don't make sense now that we split equality from ordering. It doesn't make sense to declare an `operator <=>` that returns one of these – they just add needless complexity.

and [CA 173](#):

With the separation of `<=>` and `==`, `weak_equality` has lost its primary use (of being a potential return type of `<=>`). Currently `weak_equality` serves no useful purpose in the standard (i.e., nothing in `std` acts on it), and just causes confusion (what's the difference between weak and strong, when should I use which?) The difference between the two is ill-defined (involving substitutability and “salient” properties, which are also vaguely defined). The best definition of equality for a type is the type's own `==` operator. We should not try to sub-divide the concept of equality.

The first of these comments subsumes the other, and this paper provides the wording for that change.

## § 2 Wording

---

Change 7.6.8 [expr.spaceship], paragraph 7, to remove the ability to call `<=>` on function pointers, pointers to members, and `nullptr_t`.

7 ~~If the composite pointer type is a function pointer type, a pointer-to-member type, or `std::nullptr_t`, the result is of type `std::strong_equality`; the result is `std::strong_equality::equal` if the (possibly converted) operands compare equal (`[expr.eq]`) and `std::strong_equality::nonequal` if they compare unequal, otherwise the result of the operator is unspecified.~~

Change 7.6.8 [expr.spaceship], paragraph 10:

- 10 The ~~five~~ **three** comparison category types (the types `std::strong_ordering`, ~~`std::strong_equality`~~, `std::weak_ordering`, ~~`std::weak_equality`~~, and `std::partial_ordering`) are not predefined; [...]

Change 11.11.1 [class.compare.default], paragraph 4:

- 4 A type `C` has *strong structural equality* if, given a glvalue `x` of type `const C`, either:
- (4.1) — `C` is a non-class type and `x <=> x` is a valid expression of type `std::strong_ordering` ~~or `std::strong_equality`~~, or
  - (4.2) — `C` is a class type where all of the following hold: [...]

Remove the `xxx_equality` cases from 11.11.3 [class.spaceship], paragraph 1:

- 1 The *synthesized three-way comparison* for comparison category type `R` ([`cmp.categories`]) of glvalues `a` and `b` of the same type is defined as follows:

- (1.1) — [...]
- (1.5) — Otherwise, if `R` is `partial_ordering`, then

```
a == b ? partial_ordering::equivalent :
a < b  ? partial_ordering::less      :
b < a  ? partial_ordering::greater   :
        partial_ordering::unordered
```

- (1.6) — ~~Otherwise, if `R` is `strong_equality`, then `a == b` is `strong_equality::equal` if `strong_equality::nonequal`~~
- (1.7) — ~~Otherwise, if `R` is `weak_equality`, then `a == b` is `weak_equality::equivalent` if `weak_equality::nonequivalent`~~
- (1.8) — Otherwise, the synthesized three-way comparison is not defined.

Remove the `xxx_equality` cases from 11.11.3 [class.spaceship], paragraph 3:

The *common comparison type* `U` of a possibly-empty list of `n` types `T0`, `T1`, ..., `Tn-1` is defined as follows:

- (4.1) — If any `Ti` is not a comparison category type ([`cmp.categories`]), `U` is void.
- (4.2) — ~~Otherwise, if at least one `Ti` is `std::weak_equality`, or at least one `Ti` is `std::strong_equality` and at least one `Tj` is `std::partial_ordering` or `std::weak_ordering`, `U` is `std::weak_equality` (`{cmp.weakeq}`).~~
- (4.3) — ~~Otherwise, if at least one `Ti` is `std::strong_equality`, `U` is `std::strong_equality` (`{cmp.strongeq}`).~~

- (4.4) — Otherwise, if at least one  $T_i$  is `std::partial_ordering`,  $U$  is `std::partial_ordering ([cmp.partialord])`.
- (4.5) — Otherwise, if at least one  $T_i$  is `std::weak_ordering`,  $U$  is `std::weak_ordering ([cmp.weakord])`.
- (4.6) — Otherwise,  $U$  is `std::strong_ordering ([cmp.strongord])`.

Change the example in 11.11.4 [class.rel], paragraph 3, to use a different type that has no `<:`

```
+ struct HasNoLessThan { };

struct C {
- friend std::strong_equality operator<=>(const C&, const C&);
+ friend HasNoLessThan operator<=>(const C&, const C&);
    bool operator<(const C&) const = default;           // OK,
function is deleted
};
```

Remove `<=>` from 12.7 [over.built], paragraph 19:

- 19 For every  $T$ , where  $T$  is a pointer-to-member type or `std::nullptr_t`, there exist candidate operator functions of the form:

```
bool operator==(T, T);
bool operator!=(T, T);
- std::strong_equality operator<=>(T, T);
```

Change 13.2 [temp.param]/4 to add back the bullets that [P0732R2] removed, now that these other types no longer have strong structural equality:

- 4 A non-type template-parameter shall have one of the following (optionally cv-qualified) types:
- (4.1) — a literal type that has strong structural equality ([class.compare.default]),
  - (4.2) — an lvalue reference type,
  - (4.3) — a type that contains a placeholder type ([dcl.spec.auto]), ~~or~~
  - (4.4) — a placeholder for a deduced class type ([dcl.type.class.deduct])~~;~~,
  - (4.5) — `pointer to object or pointer to function`,
  - (4.6) — `pointer to member`, or
  - (4.7) — `std::nullptr_t`.

Remove the `xxx_equality` types from the compare synopsis in 17.11.1 [compare.syn]:

```
namespace std {
    // [cmp.categories], comparison category types
- class weak_equality;
- class strong_equality;
    class partial_ordering;
```

```

class weak_ordering;
class strong_ordering;

// named comparison functions
- constexpr bool is_eq (weak_equality cmp) noexcept { return cmp == 0; }
- constexpr bool is_neq (weak_equality cmp) noexcept { return cmp != 0; }
+ constexpr bool is_eq (partial_ordering cmp) noexcept { return cmp == 0; }
+ constexpr bool is_neq (partial_ordering cmp) noexcept { return cmp != 0; }
constexpr bool is_lt (partial_ordering cmp) noexcept { return cmp < 0; }
constexpr bool is_lteq (partial_ordering cmp) noexcept { return cmp <= 0; }
constexpr bool is_gt (partial_ordering cmp) noexcept { return cmp > 0; }
constexpr bool is_gteq (partial_ordering cmp) noexcept { return cmp >= 0; }
}

```

Change 17.11.2.1 [cmp.categories.pre], paragraphs 1-2:

- 1 The types ~~weak\_equality~~, ~~strong\_equality~~, partial\_ordering, weak\_ordering, and strong\_ordering are collectively termed the comparison category types. Each is specified in terms of an exposition-only data member named value whose value typically corresponds to that of an enumerator from one of the following exposition-only enumerations:

```

enum class eq { equal = 0, equivalent = equal,
               nonequal = 1, nonequivalent = nonequal }; //
exposition only
enum class ord { less = -1, greater = 1 }; //
exposition only
enum class ncmp { unordered = -127 }; //
exposition only

```

- 2 [ Note: The types ~~strong\_ordering~~ and ~~weak\_equality~~ corresponds, respectively, to the terms ~~total ordering~~ and ~~equivalence~~ in mathematics. — end note ]

Remove 17.11.2.2 [cmp.weakeq] (the subclause that defines std ~~weak\_equality~~).

Remove 17.11.2.3 [cmp.strongeq] (the subclause that defines std ~~strong\_equality~~).

Remove the conversion operator to weak\_equality from 17.11.2.4 [cmp.partialord]:

```

namespace std {
    class partial_ordering {
        [...]

        - // conversion
        - constexpr operator weak_equality() const noexcept;

        [...]
    };
}

```

~~constexpr~~ ~~operator weak\_equality~~ ~~()~~ ~~const~~  
~~noexcept;~~

2 ~~Returns: value == 0 ? weak\_equality~~  
~~equivalent :: weak\_equality~~  
~~non-equivalent :: nonequivalent.~~ [ Note: The result is independent of the is\_ordered member. — end note ]

Remove the conversion operator to weak\_equality from 17.11.2.5 [cmp.weakord]:

```
namespace std {
    class weak_ordering {
        [...]

        // conversion
        - constexpr operator weak_equality() const noexcept;
        constexpr operator partial_ordering() const noexcept;

        [...]
    };
}
```

~~constexpr~~ ~~operator weak\_equality~~ ~~()~~ ~~const~~  
~~noexcept;~~

2 ~~Returns: value == 0 ? weak\_equality~~  
~~equivalent :: weak\_equality~~  
~~non-equivalent :: nonequivalent.~~

Remove the conversion operators to xxx\_equality from 17.11.2.6 [cmp.strongord]:

```
namespace std {
    class strong_ordering {
        [...]

        // conversions
        - constexpr operator weak_equality() const noexcept;
        - constexpr operator strong_equality() const noexcept;
        constexpr operator partial_ordering() const noexcept;
        constexpr operator weak_ordering() const noexcept;

        [...]
    };
}
```

~~constexpr~~ ~~operator weak\_equality~~ ~~()~~ ~~const~~  
~~noexcept;~~

2 ~~Returns: value == 0 ? weak\_equality~~  
~~equivalent :: weak\_equality~~  
~~non-equivalent :: nonequivalent.~~

~~constexpr~~ ~~operator strong\_equality~~ ~~()~~ ~~const~~  
~~noexcept;~~

3 ~~Returns: value == 0 ? strong\_equality~~  
~~strong\_equality~~  
~~non-equal :: nonequal.~~

Simplify the three-way comparable concepts in 17.11.4 [cmp.concept]:

```
template <typename T, typename Cat = partial_ordering>
concept three_way_comparable =
    weakly-equality-comparable-with<T, T> &&
    - (!convertible_to<Cat, partial_ordering> || partially-ordered-
with<T, T>) &&
    + partially-ordered-with<T, T> &&
    requires(const remove_reference_t<T>& a,
              const remove_reference_t<T>& b) {
        { a <== b } -> compares-as<Cat>;
    };
```

- 2 Let a and b be lvalues of type `const remove_reference_t<T>`. T and Cat model `three_way_comparable<T, Cat>` only if:

(2.1) — `(a <== b == 0)` == `bool(a == b)`.

(2.2) — `(a <== b != 0)` == `bool(a != b)`.

(2.3) — `((a <== b) <== 0)` and `(0 <== (b <== a))` are equal.

(2.4) — ~~If Cat is convertible to `strong_equality`, T models `equality_comparable_with` ([concept.equalitycomparable]).~~

(2.5) — ~~If Cat is convertible to `partial_ordering`:~~ [ Editor's note: Make the following subbullets into normal bullets ]

(2.5.1) — `(a <== b < 0)` == `bool(a < b)`.

(2.5.2) — `(a <== b > 0)` == `bool(a > b)`.

(2.5.3) — `(a <== b <= 0)` == `bool(a <= b)`.

(2.5.4) — `(a <== b >= 0)` == `bool(a >= b)`.

(2.5.5) — If Cat is convertible to `strong_ordering`, T models `totally_ordered` ([concept.totallyordered]).

```
template <typename T, typename U,
          typename Cat = partial_ordering>
concept three_way_comparable_with =
    weakly-equality-comparable-with<T, U> &&
    - (!convertible_to<Cat, partial_ordering> || partially-ordered-
with<T, U>) &&
    + partially-ordered-with<T, U> &&
```

```

three_way_comparable<T, Cat> &&
three_way_comparable<U, Cat> &&
common_reference_with<const remove_reference_t<T>&, const
remove_reference_t<U>&> &&
three_way_comparable<
common_reference_t<const remove_reference_t<T>&, const
remove_reference_t<U>&>,
Cat> &&
requires(const remove_reference_t<T>& t,
const remove_reference_t<U>& u) {
{ t <=> u } -> compares-as<Cat>;
{ u <=> t } -> compares-as<Cat>;
};

```

3 Let t and u be lvalues of types

remove\_reference\_t<T> and const remove\_reference\_t<U>, respectively. Let C be common\_reference\_t<const remove\_reference\_t<T>&, const remove\_reference\_t<U>&>. T, U, and Cat model ThreeWayComparableWith<T, U, Cat> only if:

(3.1) — t <=> u and u <=> t have the same domain.

(3.2) — ((t <=> u) <=> 0) and ((0 <=> u) <=> (u <=> t)) are equal.

(3.3) — (t <=> u == 0) == bool (t == u).

(3.4) — (t <=> u != 0) == bool (t != u).

(3.5) — Cat (t <=> u) == Cat (C (t) <=> C (u)).

(3.6) — ~~If Cat is convertible to strong equality, T and U model equality\_comparable\_with<T, U> ([concepts.equalitycomparable]).~~

(3.7) — ~~If Cat is convertible to partial ordering:~~ [ Editor's note: Make the following subbullets into normal bullets ]

(3.7.1) — (t <=> u < 0) == bool (t < u)

(3.7.2) — (t <=> u > 0) == bool (t > u)

(3.7.3) — (t <=> u <= 0) == bool (t <= u)

```
(3.7.4) — (t <=> u >= 0) ==
          bool (t >= u)
```

(3.8) — If Cat is convertible to strong\_ordering, T and U model  
totally\_ordered\_with <T, U > ([concepts.totallyordered]).

Change the root comparison category in some of the iterator operator <=>s from weak\_equality to partial\_ordering (that is, just remove the provided argument) in 23.2 [iterator.synopsis]:

```
#include <concepts>

namespace std {
    [...]

    - template<class Iterator1, three_way_comparable_with<Iterator1,
weak_equality> Iterator2>
    + template<class Iterator1, three_way_comparable_with<Iterator1>
Iterator2>
        constexpr compare_three_way_result_t<Iterator1, Iterator2>
        operator<=>(const reverse_iterator<Iterator1>& x,
                    const reverse_iterator<Iterator2>& y);

    [...]

    - template<class Iterator1, three_way_comparable_with<Iterator1,
weak_equality> Iterator2>
    + template<class Iterator1, three_way_comparable_with<Iterator1>
Iterator2>
        constexpr compare_three_way_result_t<Iterator1, Iterator2>
        operator<=>(const move_iterator<Iterator1>& x,
                    const move_iterator<Iterator2>& y);

    [...]
}
```

And the same in 23.5.1.7 [reverse.iter.cmp]:

```
- template<class Iterator1, three_way_comparable_with<Iterator1,
weak_equality> Iterator2>
+ template<class Iterator1, three_way_comparable_with<Iterator1>
Iterator2>
    constexpr compare_three_way_result_t<Iterator1, Iterator2>
    operator<=>(const reverse_iterator<Iterator1>& x,
                const reverse_iterator<Iterator2>& y);
```

13 Returns: y .base () <=> x .base ().

And the same in 23.5.3.7 [move.iter.pop.cmp]:

```
- template<class Iterator1, three_way_comparable_with<Iterator1,
weak_equality> Iterator2>
+ template<class Iterator1, three_way_comparable_with<Iterator1>
Iterator2>
```

```
constexpr compare_three_way_result_t<Iterator1, Iterator2>
operator<=>(const move_iterator<Iterator1>& x,
           const move_iterator<Iterator2>& y);
```

13 *Returns:* `x` `.base` `()` `<=>` `y` `.base` `()`.

## § 3 References

---

[P0732R2] Jeff Snyder, Louis Dionne. 2018. Class Types in Non-Type Template Parameters.  
<https://wg21.link/p0732r2>